# CS 6158 Midterm Progress Report

Budget-Constrained Selective Runtime Monitoring

Shiyun Cheng Gengkun Shao

October 19, 2026

## 1 Research Goal and Progress

Our project asks how to allocate a fixed budget among runtime monitor-creation locations using historical execution frequency, cost, useful traces, violation reports, and redundancy. The primary research question is whether budget-aware selection preserves more violations than simple heuristics or *Valg*, an adaptive online monitor-creation policy, at comparable *realized* overhead. We originally proposed evaluation on held-out executions, including unique-trace and violation recall, runtime and memory, with software evolution and exploration as extensions.

At midterm, we have **implemented five selection baselines and a reproducible evaluation pipeline**, and performed **offline selection experiments on historical TraceMOP monitor-profile data for two real Java projects**. Separately, we have a preliminary Valg reproduction dashboard comparing ValgJ and JavaMOP on four Java projects, with a partial ValgT/TraceMOP comparison. These are *distinct experiments*: our budgeted policies have not yet been deployed in the Valg runtime or compared against Valg at matched actual cost.

## 2 Implemented Modules and Methods

The Python selection pipeline reads per-group historical information and produces selected group IDs, budget usage, recall metrics, CSV tables, and plots. Five strategies are implemented: **random** feasible selection (30 draws per budget in the real-trace evaluation); **frequency ranking**; **standalone utility/cost ranking**; **redundancy-aware greedy**, which maximizes incremental distinct-trace gain per unit cost; and **additive knapsack**, which maximizes sum of standalone trace counts using a discretized dynamic program. The knapsack surrogate can double-count traces shared by several groups; it is not an exact optimum for distinct-union coverage.

For historical trace IDs $T_i$ attached to a candidate group $i$, a selected set $S$, and a proxy cost $c_i$, the current offline experiment solves the approximation

$$\max_{S} \quad \left|\bigcup_{i\in S} T_i\right| \quad \text{s.t.} \quad \sum_{i\in S} c_i \le B, \qquad \text{TraceRecall}(S) = \frac{|\cup_{i\in S} T_i|}{|\cup_i T_i|}. \tag{1}$$

Here $c_i$ is the workbook's `cost_proxy_event_checks`, *not execution time*. Candidate rows represent inferred location-by-specification groups; they are not yet verified as independently switchable monitor-creation sites. **Real-trace violation weighting is not supported yet** because the workbook lacks reliable location-attributed violation sets.

## 3 Experiments and Preliminary Results

**Historical real-trace offline evaluation.** We processed the partner-provided aggregation for `javadbf` (153 location-by-specification groups, 278 observed distinct trace IDs) and `jlemmagen` (66 groups, 2,115 trace IDs). Budgets were 10, 20, 30, 50, and 80% of summed event-check proxies. Random selection used 30 draws per budget. Table **??** shows 20% results; the denominator is the union of trace IDs *already present in historical records*. The cost-aware approaches preserved most of the previously observed distinct traces under this proxy budget. Greedy and static utility/cost tied on both reported workloads, so these results *do not establish* a benefit from explicitly modeling redundancy. Knapsack can fare worse because its additive objective rewards overlapping traces repeatedly. Frequency ranking is unstable across budgets: static-order packing may select an expensive early group at a larger

Table 1: Offline unique-trace recall at 20% event-check proxy budget (not actual runtime).

| Strategy | `javadbf` | `jlemmagen` |
|---|---|---|
| Random (30-draw mean) | 86.51% | 65.56% |
| Frequency | 44.96% | 88.84% |
| Utility/cost | 97.84% | 99.67% |
| Redundancy-aware greedy | 97.84% | 99.67% |
| Additive knapsack | 89.57% | 99.57% |

budget and thereby exclude numerous smaller groups, making independently computed selections non-nested and recall non-monotone. These outcomes are exploratory and depend on the historical aggregation.

**Valg baseline work.** The partner's reproduction dashboard covers `javadbf`, `jlemmagen`, `route53-infima`, and `xeger` using no-RV, ValgJ and JavaMOP configurations. The comparison provides a concrete environment for future realized-cost baselines. A trace-oriented ValgT/TraceMOP paired result is available for `jlemmagen`, but coverage and timing measurements are incomplete across other projects. These baseline runs *cannot yet be directly compared* with offline proxy-budget choices.

# 4 Challenges, Risks, and What We Learned

First, **violation attribution is missing**: historical reports identify violations, but do not show whether each proposed location would independently detect them. Second, **proxy cost is not realized overhead**: global instrumentation, shared monitoring work, and interactions between selected groups can invalidate an additive runtime interpretation. Third, the inferred creation location and specification group may not match a controllable creation-site switch; this needs inspection of the monitoring implementation. Fourth, all current real-trace data are from historical execution logs, without a held-out evaluation split. We cannot yet claim generalization, measured memory savings, or superiority over Valg.

# 5 Schedule and Deliverables

We remain on track for the implemented selection-pipeline deliverable; a fully realized-cost-matched Valg comparison remains the main risk. **Oct. 19–25:** validate group IDs and extraction semantics; reproduce real-trace offline plots and curate workloads. **Oct. 26–Nov. 1:** implement or test enable/disable of selected monitor-creation sites and collect per-site violation associations. **Nov. 2–8:** profile representative configurations with repeated runtime and memory measurements. **Nov. 9–15:** compare budgeted policies and Valg at similar measured overhead using held-out executions. **Nov. 16–end of term:** robustness analyses, possible revision-transfer/exploration study, final repository and report.

We propose that Shiyun maintain the selection/evaluation pipeline and Gengkun coordinate Valg profiling and instrumentation; the division requires team confirmation. **Fallback:** if switchable monitors or incremental timing measurements are infeasible, we will report a carefully labeled offline proxy-cost benchmark, explore alternative selective instrumentation, and explicitly narrow claims rather than treating proxy budgets as runtime savings.